

Week 3: Python Programming II

Objects, Errors, Vectorization

Mastery Challenge: TartanBank Account Management System

Context. You have just joined **TartanBank**, a small digital bank, as a junior software engineer. The bank is building the software that every banking system rests on: a way to represent accounts, enforce the rules of money, and manage thousands of accounts together.

The rules of money are strict and unforgiving. You cannot withdraw more than you have. You cannot deposit a negative amount. A savings account earns interest; a current account may dip into an overdraft, but only so far. If your software lets any of these rules slip, real people lose real money, so the system must be built carefully, with each rule enforced inside the objects themselves.

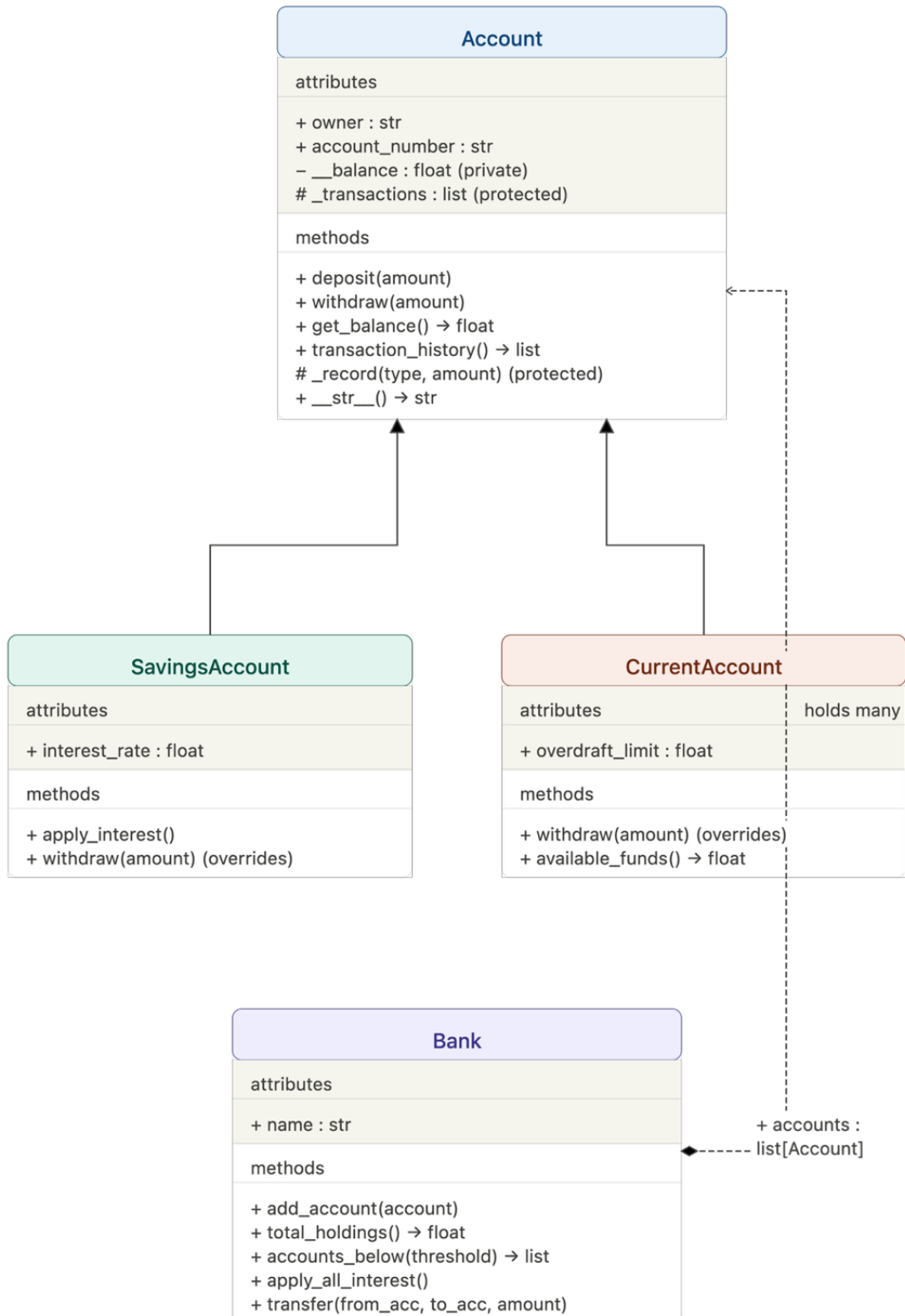
The Class Diagram

The whole system is four classes. Keep this diagram beside you throughout: every attribute and method you are asked to write appears here. **Account** is the base class. **SavingsAccount** and **CurrentAccount** inherit from it. **Bank** holds a collection of accounts.

Note: The account balance is **private** for a reason you will discover in Level 1

Legend

+ public
 # protected (single underscore)
 - private (double underscore)
 —> inherits from



Progression and Rationale

You will build this system in **six levels**. Each level introduces one object-oriented concept and builds directly on the previous one. By the end you will have a working bank with three kinds of account and an interactive menu to operate it.

Level	What it demonstrates
Level 1	Classes, Attributes & Encapsulation: Proves you can hide state and expose controlled interfaces.
Level 2	Error Handling: Demonstrates failing loudly and safely by raising exceptions on invalid state operations.
Level 3	Inheritance & <code>super()</code>: Shows you can extend a base class cleanly to model a Savings Account.
Level 4	Method Overriding & Polymorphism: Proves you can change inherited behavior to model an overdraft Current Account.
Level 5	Composition & Vectorization: Demonstrates managing a collection of objects and aggregating data efficiently.
Level 6	Putting It All Together: Proves you can wire your OOP back-end to a user-facing interactive menu.

Deliverables

Submit **both** of the following on Canvas.

1. **Code.** Your `.ipynb` notebook with every ‘# YOUR CODE HERE’ section filled in. It must run from top to bottom without errors.
2. **Report (one to two pages).** A short write-up containing the printed output of each question, one or two sentences explaining a design decision for each question, and written answers to all six Logic Questions.

3.3 Scope

Pure Python only. No ‘pandas’, ‘lambda’, ‘zip’, or ‘enumerate’ allowed. Use named functions and plain loops or comprehensions instead.

TartanBank Mission

Build an object-oriented banking system that enforces financial rules, leverages inheritance, and handles errors safely.

Complete the six levels in order. Each level introduces a new software engineering concept and builds on the previous level.

Level 1: Build the Account

XP: 10

Tier: Foundation

Estimated Time: 15 - 20 mins

Idea. *Every bank starts with a single account that encapsulates its balance.*

Task. Complete the `Account` class.

- `owner` & `account_number`: Public attributes.
- `__balance`: Private attribute. No code outside the class should be able to read or change it directly.
- `_transactions`: A protected list that records each transaction as a `(kind, amount)` pair.
- `deposit(amount)`: Adds money. Reject any amount below the minimum deposit.
- `withdraw(amount)`: Removes money. Reject zero/negative amounts, and reject withdrawals larger than balance.
- `get_balance()`: The only way to read the balance from outside.
- `transaction_history()`: Returns a copy of the transactions list.
- `__str__()`: Returns a clear one-line summary of the account.

If You're Stuck

Review encapsulation and private attributes. Think about which methods should be the only places that modify the balance.

Logic Question 1: Why make `__balance` private instead of public? What could go wrong if any code could simply set `account.balance = -5000` directly?

Topics Covered: Classes, Attributes, Encapsulation

Level 2: Enforce the Rules

XP: 20

Tier: Core

Estimated Time: 15 - 20 mins

Idea. *When something is not allowed, fail loudly and safely by raising an error.*

What to build

1. Confirm your `Account` class raises a `ValueError` for each of these:
 - A negative opening balance
 - A deposit below the minimum
 - A zero or negative withdrawal
 - A withdrawal larger than the balance.
2. Write a function `safe_withdraw(account, amount)` that tries to withdraw and uses `try / except` to catch a `ValueError`. Instead of crashing, it prints a friendly message. It returns `True` if the withdrawal succeeded and `False` if it failed.

If You're Stuck

Go over error handling: specifically, the `raise` statement and the `try / except` block. Catch the specific error type (`except ValueError`) rather than catching everything with a bare `except` that way you only handle the errors you expect.

Logic Question 2: Why is it better to raise an error inside `withdraw` than to silently do nothing when there are not enough funds?

Topics Covered: Error handling, `raise`, `try / except`

Level 3: The Savings Account

XP: 30

Tier: Core

Estimated Time: 20 - 30 mins

Idea. *Inherit from Account and add interest logic.*

Task. Complete the `SavingsAccount` class, which inherits from `Account`:

- `interest_rate` attribute (for example 0.05 for 5%). Reject a negative rate.
- Use `super().__init__()` inside the constructor to reuse the parent's setup. Do not rewrite the owner or balance logic.
- `apply_interest()` works out the interest on the current balance and deposits it. Reuse the existing `deposit` method rather than touching the balance directly.
- Override `__str__()` so the summary shows this is a savings account and its interest rate.

If You're Stuck

Go over inheritance and the `super()` function. Declaring `class SavingsAccount(Account)` gives you every method from `Account` for free. `super().__init__()` runs the parent's constructor so you don't repeat yourself. If interest does not appear in the transaction history, check that you reused `deposit`.

Logic Question 3: Your `apply_interest` calls `self.deposit(...)` rather than changing the balance directly. Why is reusing `deposit` the better choice?

Topics Covered: Inheritance, `super()`, method reuse

Level 4: The Current Account

XP: 40

Tier: Core / Extension

Estimated Time: 20 - 30 mins

Idea. *Override methods to allow controlled overdrafts.*

Task. Complete the `CurrentAccount` class, which inherits from `Account`:

- `overdraft_limit` attribute (a positive number).
- `available_funds()` returns the balance plus the overdraft limit (how much can still be withdrawn).
- Override `withdraw(amount)` to allow the balance to go negative, but never below the overdraft limit. Still reject zero/negative amounts.
- Override `__str__()` to show this is a current account and its overdraft limit.

If You're Stuck

Go over method overriding and polymorphism. To change the balance safely from inside the subclass, use the protected helper provided in the template rather than touching the private balance directly.

Logic Question 4: Both `Account` and `CurrentAccount` have a `withdraw` method. When you call `withdraw` on a list of mixed accounts, how does Python know which version to run, and why is that useful?

Topics Covered: Method overriding, polymorphism

Level 5: The Bank

XP: 50

Tier: Extension

Estimated Time: 30 - 40 mins

Idea. *Manage a collection of accounts using composition and collection processing.*

Task. Complete the Bank class:

- `name` and `accounts` (an empty list to begin with).
- `add_account(account)` adds any kind of account to the list.
- `total_holdings()` the sum of every account's balance, written with a comprehension and `sum()`.
- `accounts_below(threshold)` a list of every account whose balance is below the given amount, written as a comprehension.
- `apply_all_interest()` applies interest to every savings account. Only use `isinstance` to pick them out.
- `transfer(from_acc, to_acc, amount)` withdraws from one account and deposits into another.

If You're Stuck

Go over composition and vectorization with list comprehensions. A comprehension like `[a for a in self.accounts if ...]` filters a list in one line; `sum(a.get_balance() for a in self.accounts)` aggregates one. Remember to initialise accounts inside the constructor, not at the class level.

Logic Question 5: `total_holdings` uses a comprehension instead of a manual loop with a running total. In a bank with two million accounts, why does writing it as an operation over the whole collection matter?

Topics Covered: Composition, list comprehensions, collection processing, `isinstance`

Level 6: The Interactive Bank

XP: 60

Tier: Capstone

Estimated Time: 40 - 50 mins

Idea. *Connect your banking classes to a user-facing interface.*

Task. Complete `main(bank)` so it loops, showing a menu with at least these options:

1. View all accounts
2. Deposit to an account (chosen by account number)
3. Withdraw from an account (use your `safe_withdraw` function)
4. Transfer between two accounts
5. Apply interest to all savings accounts
6. Show the bank's total holdings
7. Exit

The menu must handle mistakes gracefully: an unknown account number or a non-numeric amount should print a message, not crash the program.

If You're Stuck

Go over `while` loops, the `input()` function, and how to call methods on objects. If you find yourself writing balance arithmetic inside `main` stop, that logic belongs in the account or bank methods you already wrote. `main` only needs to ask the user what they want and call the right method.

Logic Question 6: Why is it useful to keep the banking logic inside classes such as `Account`, `SavingsAccount`, and `Bank` instead of placing all of the logic directly inside the menu code?

Topics Covered: `while` loops, `input()`, composition

Level Bonus: Advanced OOP

XP: –

Tier: Optional

Estimated Time: 45 - 60 mins

Dunder methods, `@property`, `@classmethod`, abstract base classes

Idea. *Extend the system with a new account type without changing what you have already built.*

This section is self-contained. Do not change any class from Levels 1–6. You will add one new account type, `CreditAccount`, that uses four advanced features. A credit account models a credit card: its balance represents the amount **owed**. You **charge** it to borrow (up to a credit limit) and **pay** it to reduce the debt. Leaving the original classes untouched while adding new behaviour is itself a design principle.

1. **Dunder methods.** Add three special methods:
 - `__eq__` so two accounts are equal when they share an account number.
 - `__lt__` so one account is “less than” another by how much it owes (this is what `sorted()` order them).
 - `__len__` so `len(account)` returns its number of transactions.
2. **Properties.** Add two `@property` values:
 - `available_credit`, the credit limit minus what is owed.
 - `is_over_limit`, true when the debt exceeds the limit.
 - Both are computed fresh each time and cannot be assigned to.
3. **Alternative constructor.** Add a `@classmethod`:
 - Call it `from_record(cls, record)`.
 - It takes a string like `"CR001,Ada,5000"`, splits it, and returns a fully built `CreditAccount`.
4. **An interface.** Implement the `Reportable` contract:
 - A small `Reportable` abstract base class is given to you in the template; it declares an abstract `monthly_statement()`.
 - Make `CreditAccount` inherit from it and implement `monthly_statement()`.
 - The statement should report what is owed, the available credit, and the interest due this month ($\text{owed} \times \text{APR} \div 12$).

If You're Stuck

Go over dunder (magic) methods, the `@property` and `@classmethod` decorators, and abstract base classes (the `abc` module). When you write `a == b`, Python calls `a.__eq__(b)`. A `@property` is accessed like `account.available_credit`, with no brackets. A `@classmethod` receives the class as `cls` and builds an instance with `cls(...)`. An `@abstractmethod` forces every subclass to implement

it, the contract is checked the moment an object is created.

Bonus Logic Question: Python has no `interface` keyword. How does an abstract base class with `@abstractmethod` achieve the same thing, and why is that useful in a system with many account types?